VALIDATION REPORT

Development of an Ethernet-Based Vehicle Monitoring System Using Linux Socket Programming

1. Objective

The objective of this validation is to test the communication between the Vehicle Electronic Control Unit (Vehicle ECU) and the Dashboard Electronic Control Unit (Dashboard ECU) using TCP and UDP socket programming in Linux.

The system generates simulated vehicle parameters, including vehicle speed, engine RPM, and steering angle. These parameters are transmitted from the Vehicle ECU to the Dashboard ECU and displayed at the receiving end.

2. Test Environment

Parameter	Description
Operating System	Ubuntu Linux
Programming Language	C
Compiler	GCC
Packet Analysis Tool	Wireshark
Network Interface	Loopback (`lo`)
IP Address	127.0.0.1
Server Port	5000
Communication Protocols	TCP and UDP

3. Validation Methodology

The system was tested by executing the Vehicle ECU and Dashboard ECU programs in separate terminal windows. TCP and UDP communication were tested independently.

Wireshark was used to capture and inspect the packets exchanged between the applications. The captured traffic was examined to verify the transport protocol, source and destination ports, and transmitted vehicle data.

4. Test Cases and Results

Test Case 1: Normal TCP Communication Objective: To verify reliable connection establishment and vehicle data transmission using TCP.

Procedure: 1. Start the TCP Dashboard ECU server. 2. Execute the TCP Vehicle ECU client in another terminal. 3. Observe the received vehicle data in the Dashboard terminal.

Expected Result: The TCP connection is established, and the Dashboard ECU receives and displays vehicle speed, engine RPM, and steering angle.

Observed Result: The TCP Vehicle ECU connected to the Dashboard ECU, and vehicle data was transmitted and displayed successfully.

Status: PASSED

Test Case 2: Normal UDP Communication **Objective:** To verify vehicle data transmission using UDP.

Procedure: 1. Start the UDP Dashboard ECU. 2. Run the UDP Vehicle ECU in another terminal. 3. Observe the incoming vehicle data.

Expected Result: The Dashboard ECU receives UDP datagrams containing vehicle parameters.

Observed Result: UDP packets containing vehicle data were visible in Wireshark. The Dashboard ECU had previously displayed the received vehicle parameters.

Status: PASSED, subject to confirmation of the final test run.

Test Case 3: Dashboard ECU Not Running **Objective:** To examine application behavior when the Dashboard ECU is unavailable.

Procedure: 1. Stop the Dashboard ECU. 2. Attempt to start the TCP Vehicle ECU. 3. Observe the TCP connection result. 4. Run the UDP Vehicle ECU with the Dashboard stopped and observe its behavior.

Expected Result: The TCP connection attempt should fail when no server is listening. The UDP sender may continue sending datagrams because UDP does not require a connection to be established.

Observed Result: Record the exact TCP error and UDP sender behavior observed during testing.

Status: Update after confirming the observed results.

Test Case 4: Invalid Server IP Address **Objective:** To verify the TCP client's handling of an unreachable or unavailable destination.

Procedure: 1. Create a temporary copy of the TCP Vehicle ECU source file. 2. Change the destination address to `192.0.2.1`. 3. Compile and execute the temporary program. 4. Observe the connection result.

Expected Result: The connection should fail if the destination cannot be reached or no server is available.

Observed Result: Record the actual error message produced by the program.

Status: Update after completing the test.

Test Case 5: Continuous Data Transmission **Objective:** To verify repeated transmission of vehicle parameters over time.

Procedure: 1. Start the Dashboard ECU. 2. Start the corresponding Vehicle ECU. 3. Allow the programs to communicate continuously. 4. Observe the received data and captured packets.

Expected Result: Vehicle data should be transmitted repeatedly at approximately one-second intervals.

Observed Result: Update after completing the continuous-transmission test.

Status: Update after testing.

5. Wireshark Packet Analysis

Wireshark was used to inspect TCP and UDP traffic captured on the loopback interface.

5.1 UDP Analysis The UDP capture showed packets with the following characteristics:

- Source IP address: `127.0.0.1`
- Destination IP address: `127.0.0.1`
- Destination port: `5000`
- Transport protocol: UDP
- Application payload: Vehicle speed, engine RPM, and steering angle

The packet payload confirmed that vehicle-monitoring information was being transmitted.

5.2 TCP Analysis The TCP capture showed an established connection between the Vehicle ECU client and the Dashboard ECU server.

Observed characteristics included:

- Source IP address: `127.0.0.1`
- Destination IP address: `127.0.0.1`
- Client source port: `37170` in the captured session
- Server destination port: `5000`
- Transport protocol: TCP
- Application payload: Vehicle speed, engine RPM, and steering angle

TCP data packets and acknowledgment packets were visible in Wireshark.

6. TCP and UDP Comparison

TCP establishes a connection and provides reliable, ordered delivery of a byte stream. UDP sends independent datagrams without establishing a transport connection and does not guarantee delivery or ordering.

The experiments demonstrated the different socket operations used by the two protocols:

TCP	UDP
`SOCK_STREAM`	`SOCK_DGRAM`
`connect()`	`sendto()`
`listen()` and `accept()` on the server	`recvfrom()` on the server
`send()` and `recv()`	`sendto()` and `recvfrom()`

7. Limitations

The communication tests were conducted using the loopback interface (`127.0.0.1`). Therefore, the results validate local socket communication between applications running on the same Ubuntu machine.

These tests do not demonstrate communication between separate computers over a physical Ethernet network.

The vehicle parameters were simulated values rather than measurements obtained from an actual vehicle or automotive ECU.

8. Conclusion

The project implements TCP- and UDP-based communication between a simulated Vehicle ECU and Dashboard ECU using Linux socket programming in C.

Wireshark was used to inspect the transmitted packets and identify the application data carried by each protocol. The initial tests demonstrated successful TCP and UDP data transmission over the loopback interface.

The remaining validation scenarios should be marked as passed only after their actual outcomes have been observed and recorded.